

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN
THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO TUẦN 12

MÔN: MASSIVE DATA MINING

GitHub AI Trend Analyzer

Hệ thống khai thác và phân tích dữ liệu lớn
từ GitHub Events API theo thời gian thực

Production: github.chipthoc.com

Thành viên	Vai trò
Nguyễn Văn Linh	Ingestion pipeline, Kafka, AI Filter
Nguyễn Hải Lâm	Spark Streaming, ClickHouse, Storage
Trần Quang Minh	FastAPI, Frontend, Observability

Hà Nội, tháng 5 năm 2026

0 Mục lục

1	Giới thiệu bài toán và động lực	3
1.1	Vấn đề thực tế (Painpoint)	3
1.2	Giải pháp đề xuất	3
1.3	Quy mô dữ liệu và ràng buộc kỹ thuật	4
1.4	Các thách thức kỹ thuật cốt lõi	4
2	Phân tích phương pháp và lựa chọn giải pháp	5
2.1	Các phương án đã xem xét	5
2.2	Lý do chọn ClickHouse thay vì PostgreSQL / MySQL	5
2.3	Lý do chọn Kafka thay vì truyền trực tiếp	5
3	Kiến trúc hệ thống	6
3.1	Tổng quan kiến trúc	6
3.2	Các dịch vụ trong docker-compose	8
4	Xử lý dữ liệu lớn — Massive Data Pipeline	8
4.1	Lớp Ingestion: GitHub Events API → Kafka	8
4.2	Lớp Stream Processing: Spark Structured Streaming	9
4.3	Schema dữ liệu trong ClickHouse	12
5	Tìm kiếm ngữ nghĩa và AI Endpoints	13
5.1	Hybrid Search Pipeline	13
5.2	Các AI Endpoints	13
5.3	Lý do dùng Ollama thay vì cloud API	13
6	Observability và Độ tin cậy vận hành	14
6.1	Prometheus Metrics	14
6.2	Cron Jobs — Kiểm tra sức khỏe định kỳ	14
6.3	Grafana Dashboard	15
7	Kết quả và Demo hệ thống	15
7.1	Frontend Dashboard	15
7.2	API Endpoints	16
7.3	Số liệu vận hành thực tế	16
7.4	Luồng xử lý end-to-end	17
8	Phân tích ưu/nhược điểm và hướng phát triển	18
8.1	Ưu và nhược điểm của giải pháp hiện tại	18
8.2	Hướng phát triển	18

9 Kết luận	19
Phân công công việc	19
Tài liệu tham khảo	20

1 Giới thiệu bài toán và động lực

1.1 Vấn đề thực tế (Painpoint)

Nhà nghiên cứu, kỹ sư, và người ra quyết định trong lĩnh vực AI/ML thường gặp một câu hỏi đơn giản nhưng khó trả lời: *“Hôm nay, framework nào đang được cộng đồng quan tâm nhất?”* hay *“Tuần này topic gì đang tăng nhiệt?”*

Hiện tại, không có công cụ nào cho phép theo dõi điều này một cách **tức thời và có căn cứ dữ liệu**. Người dùng buộc phải:

- Lướt tay GitHub Trending — chỉ cập nhật hàng ngày, không có ngữ cảnh lịch sử, không lọc được theo domain AI/ML.
- Đọc bản tin kỹ thuật — thường trễ vài ngày, phụ thuộc biên tập viên, không phản ánh tín hiệu thực từ cộng đồng lập trình.
- Tự viết script poll API — mất thời gian, không scale, dữ liệu không được lưu trữ có cấu trúc để phân tích sau.

Kết quả là: **xu hướng thực sự của hệ sinh thái AI/ML bị ẩn sau hàng triệu sự kiện thô**, và không ai có thể nhìn thấy nó trong thời gian thực mà không có hạ tầng phù hợp.

1.2 Giải pháp đề xuất

GitHub AI Trend Analyzer là hệ thống khai thác và phân tích dữ liệu lớn theo thời gian thực từ GitHub Events API, cho phép bất kỳ ai truy cập dashboard tại github.chiphoc.com và nhận được câu trả lời ngay lập tức.

Bảng 1: Từ câu hỏi thực tế đến tính năng hệ thống

Câu hỏi người dùng	Tính năng đáp ứng
“Repo AI nào đang nổi nhất hôm nay?”	Top repos theo WatchEvent, cập nhật mỗi 60 giây
“Framework nào tăng stars mạnh nhất tuần này?”	Shock-movers: delta stars theo window 7 ngày
“Topic nào đang tăng tốc so với tuần trước?”	Topic rotation: so sánh tần suất giữa hai window
“Tìm repo về LLM fine-tuning”	Hybrid search: lexical + semantic (embedding)
“Tóm lược repo này cho tôi”	AI brief: LLM tóm lược có dẫn chiếu dữ liệu thực

1.3 Quy mô dữ liệu và ràng buộc kỹ thuật

- **Nguồn dữ liệu:** GitHub Events API phát sinh khoảng **2-3 triệu sự kiện mỗi ngày**, bao gồm WatchEvent, ForkEvent, PushEvent, PullRequestEvent, v.v.
- **Yêu cầu độ trễ:** Dashboard phải phản ánh sự kiện mới trong vòng **dưới 5 phút** — không chấp nhận batch hàng giờ.
- **Yêu cầu query:** Phép tính tổng hợp GROUP BY trên hàng chục triệu bản ghi phải hoàn thành trong dưới **500ms**.
- **Semantic search:** Keyword matching đơn giản không hiểu được ý định truy vấn tự nhiên — cần tầng embedding.

1.4 Các thách thức kỹ thuật cốt lõi

Bảng 2: Các thách thức kỹ thuật và hướng giải quyết

Thách thức	Mô tả	Giải pháp áp dụng
Volume	Hàng triệu events/ngày, cần lọc và chuẩn hoá liên tục	Kafka + Spark Structured Streaming
Velocity	Events đến liên tục, không theo lịch cố định	Micro-batch trigger 60s, watermark 10 phút
Variety	JSON payload không đồng nhất giữa các loại event	Schema chuẩn hoá trong Spark transform
Deduplication	Kafka at-least-once → cùng event có thể đến nhiều lần	ReplacingMergeTree trong ClickHouse
Analytics scale	Query tổng hợp trên hàng chục triệu bản ghi	ClickHouse columnar + vectorised execution
Semantic gap	Keyword search không hiểu ngữ nghĩa của truy vấn	Hybrid lexical/semantic search với bge-m3

2 Phân tích phương pháp và lựa chọn giải pháp

2.1 Các phương án đã xem xét

Trước khi chốt kiến trúc, nhóm đã đánh giá bốn hướng tiếp cận:

Bảng 3: So sánh các phương án xử lý dữ liệu lớn

Phương án	Cơ chế	Ưu điểm	Nhược điểm / Lý do loại
Batch ETL (Airflow + Postgres)	Poll theo lịch, ghi vào RDBMS	Đơn giản, dễ vận hành	Latency hàng giờ; RDBMS chậm trên phép GROUP BY lớn → không đáp ứng yêu cầu freshness
Polling DuckDB / SQLite	+ Poll trực tiếp vào file DB	Không cần infra nặng	Không concurrent; không scale khi data lớn → chỉ phù hợp PoC
Kafka + Apache Flink	True streaming, stateful ops	Fault-tolerant, latency thấp nhất	Learning curve cao; Java ecosystem khó tích hợp Python ML stack → loại do chi phí dev
Kafka + Spark Structured Streaming + ClickHouse	Micro-batch 60s, dual sink	Dedup tự động; watermark 10 phút; ClickHouse sub-second analytics; Python native	Lựa chọn của nhóm

2.2 Lý do chọn ClickHouse thay vì PostgreSQL / MySQL

- **Columnar storage:** ClickHouse lưu dữ liệu theo cột → chỉ đọc cột cần thiết trong GROUP BY, giảm I/O tới 10-100 lần.
- **Vectorised execution:** xử lý hàng triệu giá trị trong một lần CPU instruction batch.
- **MergeTree engine family:** ReplacingMergeTree tự dedup theo version column; SummingMergeTree tự cộng dồn counter khi merge.
- Benchmark nội bộ: query GROUP BY language, date trên 10M bản ghi: PostgreSQL $\approx$ 15-30s, ClickHouse $<$ 200ms.

2.3 Lý do chọn Kafka thay vì truyền trực tiếp

Kafka đóng vai trò *decoupling buffer* giữa lớp ingest và lớp xử lý:

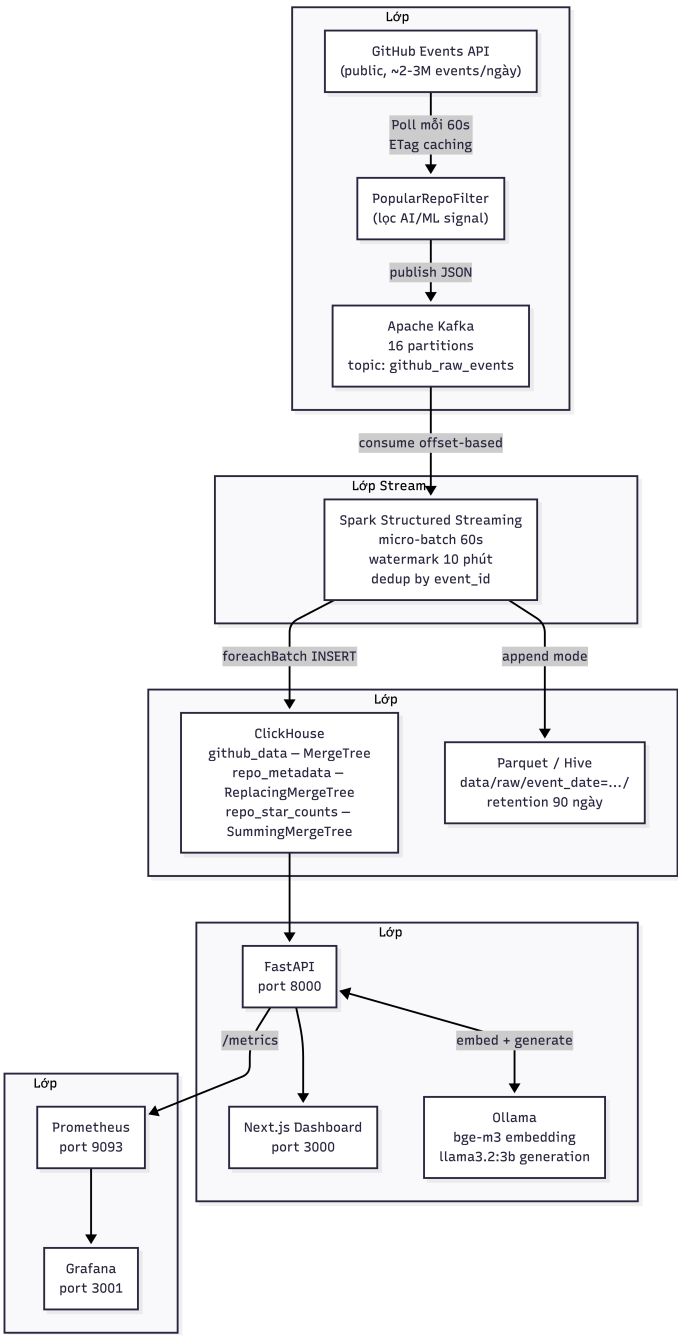
- Backpressure: nếu Spark chậm, events đợi trong Kafka thay vì mất.
- Replay: có thể tua lại offset để reprocess khi schema thay đổi.
- 16 partition $\approx$ 16 Spark task song song, khớp với số core CPU của máy chủ (Intel i7-12700F, 20 threads).

3 Kiến trúc hệ thống

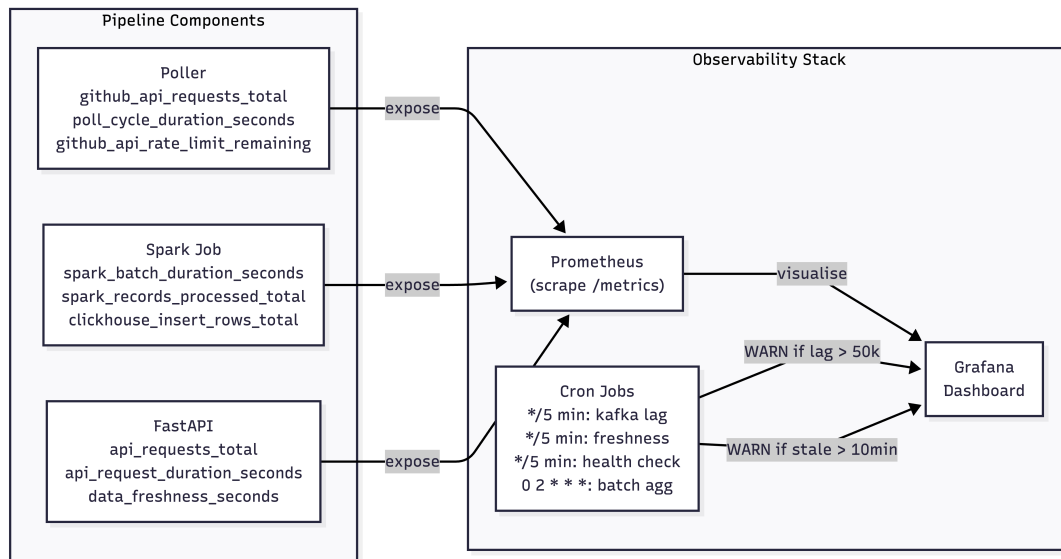
3.1 Tổng quan kiến trúc

Hệ thống được chia thành năm lớp logic rõ ràng:

1. **Ingestion:** Poll GitHub Events API $\rightarrow$ lọc qua PopularRepoFilter $\rightarrow$ publish lên Kafka.
2. **Stream Processing:** Spark Structured Streaming consume Kafka $\rightarrow$ parse JSON $\rightarrow$ dual sink.
3. **Storage:** ClickHouse (OLAP serving) + Parquet (cold archive).
4. **Serving:** FastAPI expose REST API; Ollama cung cấp embedding và generation.
5. **Observability:** Prometheus thu thập metrics $\rightarrow$ Grafana visualise; cron jobs kiểm tra health định kỳ.



Hình 1: Kiến trúc tổng thể GitHub AI Trend Analyzer



Hình 2: Luồng observability: metrics và tracing trong hệ thống

3.2 Các dịch vụ trong docker-compose

Bảng 4: Danh sách dịch vụ và cổng mạng

Dịch vụ	Cổng	Vai trò
Next.js Frontend	3000	Giao diện dashboard trực quan
FastAPI	8000	REST API, pipeline status, AI endpoints
Apache Kafka	9092	Message broker, 16 partitions
ClickHouse HTTP	8123	External tooling, SQL query
ClickHouse Native	9100	Spark JDBC connector
Prometheus	9093	Metrics scraper
Grafana	3001	Metrics dashboard, alerting
Ollama	11435	Embedding (bge-m3) + generation (llama3.2:3b)
Zookeeper	—	Kafka cluster coordination

4 Xử lý dữ liệu lớn — Massive Data Pipeline

4.1 Lớp Ingestion: GitHub Events API → Kafka

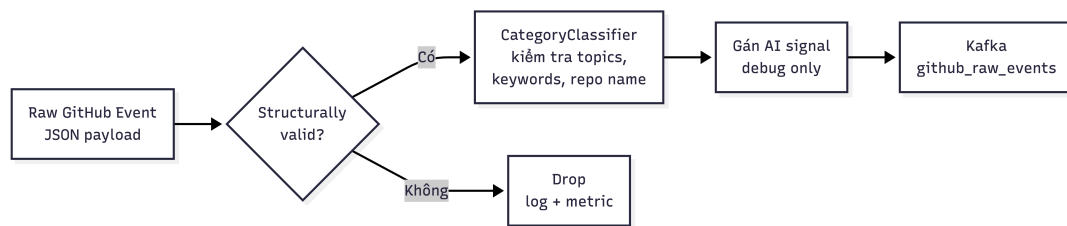
Cơ chế polling và tối ưu quota

- **Token rotation:** hỗ trợ nhiều GitHub Personal Access Token luân phiên, tự động chuyển token khi hit rate limit.
- **ETag caching:** gửi header If-None-Match trong mỗi request; nếu GitHub trả HTTP 304 Not Modified → bỏ qua, không publish lên Kafka,

không tốn quota.

- **Rate limit handling:** khi tất cả token đều cạn, hệ thống tính thời điểm reset sớm nhất và ngủ đúng thời gian đó.
- **Batch size histogram:** metric `poll_batch_size` theo dõi số event trả về mỗi lần poll.

AI/ML Event Filter



Hình 3: Pipeline lọc sự kiện AI/ML

Từ khoá AI signal bao gồm: `llm`, `machine-learning`, `deep-learning`, `neural-network`, `rag`, `embedding`, `diffusion`, `vector-database`, `multimodal`, và nhiều hơn.

Lưu ý thiết kế quan trọng: filter *không* loại bỏ event không có tín hiệu AI — mọi event hợp lệ đều đi qua Kafka để pipeline không bỏ sót repository lớn như `torvalds/linux` hay `facebook/react`. AI signal chỉ dùng cho phân loại category.

4.2 Lớp Stream Processing: Spark Structured Streaming

Cấu hình micro-batch

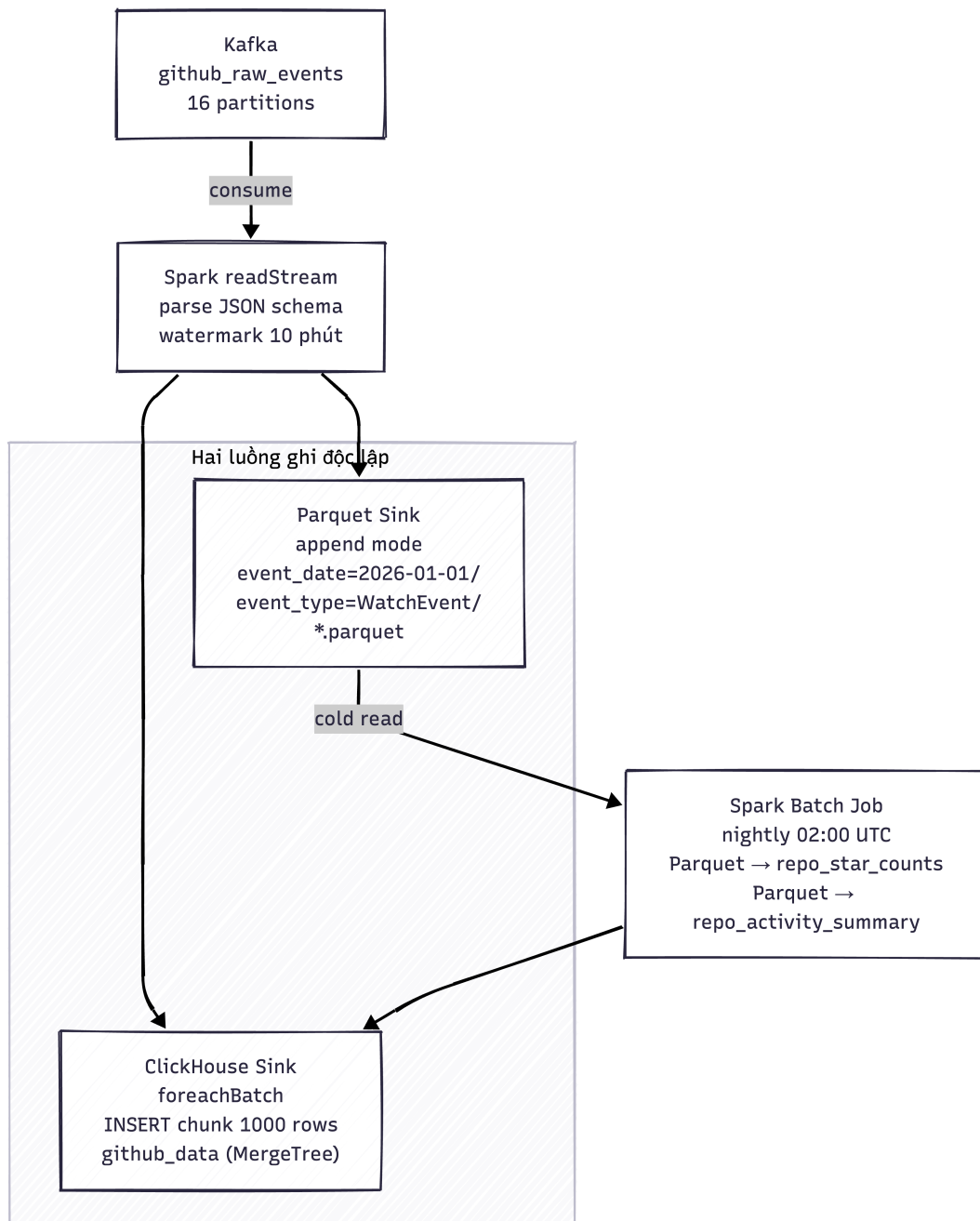
Bảng 5: Cấu hình Spark Structured Streaming

Tham số	Giá trị	Lý do
Trigger interval	60 giây	Cân bằng latency vs overhead
Watermark	10 phút	Xử lý late-arriving events
Spark master	<code>local[16]</code>	Khai thác 16 core hiệu năng của CPU
Driver memory	8 GB	Phù hợp 32 GB RAM hệ thống
Executor memory	12 GB	Headroom cho Parquet write buffer
Kafka partitions	16	Song song với số core
ClickHouse insert chunk	1.000 rows	Tránh timeout trên batch lớn

Dual-sink pattern

Spark đọc một `readStream` từ Kafka rồi fan-out thành hai luồng ghi độc lập (dual sink):

- **Parquet sink** (append mode): lưu raw event theo Hive partitioning `event_date / event_type`, retention 90 ngày. Schema cố định, phù hợp batch reprocessing.
- **ClickHouse sink** (foreachBatch): INSERT theo chunk 1.000 dòng vào bảng `github_data`. Dedup tự nhiên qua MergeTree ORDER BY (`repo_id`, `created_at`).



Hình 4: Dual-sink pattern: Kafka → Parquet + ClickHouse

4.3 Schema dữ liệu trong ClickHouse

Bảng 6: Bảng chính trong ClickHouse — github_analyzer

Bảng	Engine	Mô tả
github_data	MergeTree PARTITION BY toYYYYMM	Nguồn sự thật duy nhất: raw events + metadata snapshot tại thời điểm ingest
repo_metadata	ReplacingMergeTree (refreshed_at)	Metadata mới nhất của mỗi repository (stars, forks, topics, language)
repo_metadata_history	ReplacingMergeTree (snapshot_at)	Audit log snapshot theo từng chu kỳ fetch
repo_star_counts	SummingMergeTree	Daily star delta, tính bởi Spark batch job
repo_activity_summary	ReplacingMergeTree (computed_at)	Weekly aggregate count per repo per event type
github_data_to_repo_metadata_history_mv		Materialized View: tự động đẩy snapshot từ github_data vào history

Parquet — cold storage

Listing 1: Cấu trúc thư mục Parquet (Hive partitioning)

```

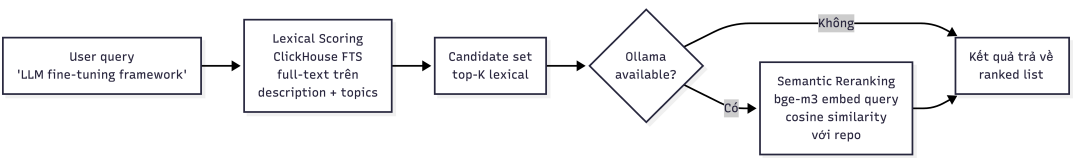
1 data/raw/|—
2   event_date=2026-01-01/| |—
3     part-00001-abc123.parquet| |—
4     part-00002-def456.parquet| |—
5   event_date=2026-01-02/| |—
6   ...| |—
7 checkpoints/ ← Spark checkpoint state
    
```

Hive partitioning cho phép *predicate pushdown*: khi query theo ngày, Spark chỉ đọc partition tương ứng, bỏ qua toàn bộ partition còn lại.

5 Tìm kiếm ngữ nghĩa và AI Endpoints

5.1 Hybrid Search Pipeline

Bài toán semantic search trên kho hàng trăm nghìn repository đòi hỏi hai tầng xử lý:



Hình 5: Pipeline tìm kiếm hybrid: lexical + semantic

5.2 Các AI Endpoints

Bảng 7: AI endpoints và chức năng

Endpoint	Model	Chức năng
GET /ai/search	bge-m3	Hybrid lexical/semantic search
GET /ai/repo-brief	llama3.2:3b	Tóm lược + lý do trending của một repo
GET /ai/repo-compar	llama3.2:3b	So sánh cấu trúc giữa hai repo
GET /ai/related-rep	bge-m3	Gợi ý repository liên quan
GET /ai/market-brie	llama3.2:3b	Tóm lược thị trường AI theo tuần

5.3 Lý do dùng Ollama thay vì cloud API

- **Không phụ thuộc internet** trong lúc demo.
- **Không tốn chi phí API** cho prototype và báo cáo.
- **Privacy:** dữ liệu GitHub không rời khỏi hệ thống.
- Giới hạn: model nhỏ hơn (3B parameters) → chất lượng generation thấp hơn GPT-4; đây là trade-off chấp nhận được cho scale hiện tại.

6 Observability và Độ tin cậy vận hành

6.1 Prometheus Metrics

Hệ thống định nghĩa **21 metric** trải đều trên 5 nhóm:

Bảng 8: Danh sách metric theo nhóm

Nhóm	Metric tiêu biểu	Ý nghĩa
GitHub API	github_api_rate_limit	Theo dõi quota còn lại theo từng token
GitHub API	github_api_token_exhausted	Cảnh báo khi token cạn
Kafka	kafka_messages_produced	Throughput vào Kafka
Spark	spark_batch_duration	Thời gian xử lý mỗi micro-batch
Spark	spark_records_processed	Tổng bản ghi đã xử lý
Storage	data_freshness_seconds	Độ trễ dữ liệu mới nhất so với hiện tại
FastAPI	api_request_duration_seconds	Latency phân vị 50/95/99
FastAPI	api_in_flight_requests	Request đang xử lý đồng thời

6.2 Cron Jobs — Kiểm tra sức khỏe định kỳ

Bảng 9: Cron jobs vận hành tự động

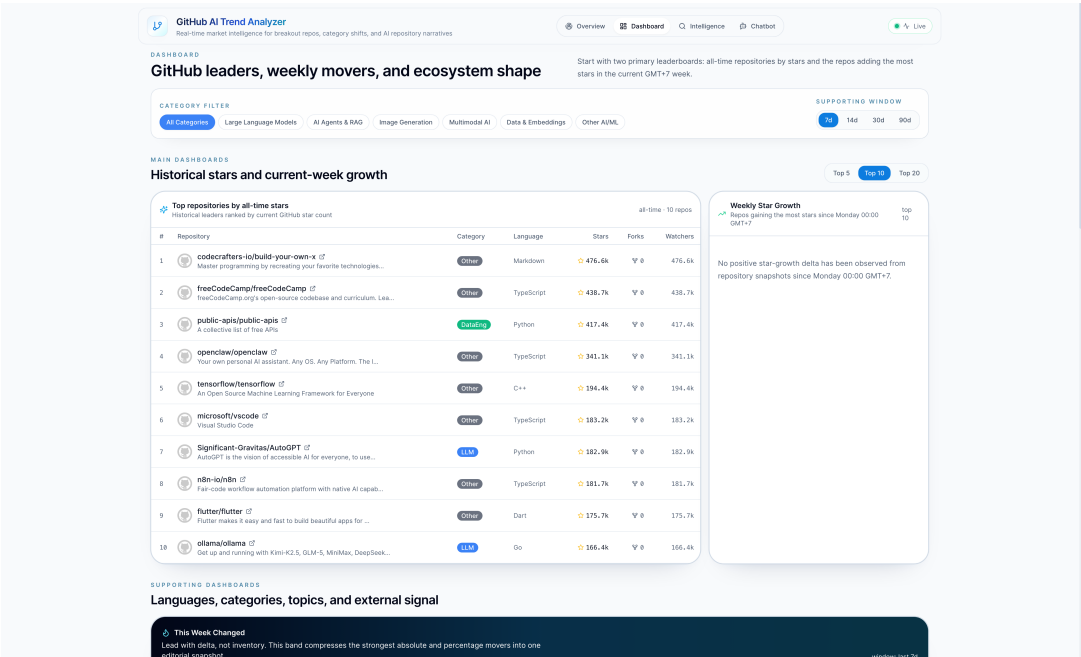
Script	Lịch (UTC)	Điều kiện cảnh báo
Kafka lag check	Mỗi 5 phút	Consumer lag > 50.000 messages
Data freshness check	Mỗi 5 phút	max(created_at) stale > 10 phút
Health check (5-point)	Mỗi 5 phút	Docker / Kafka / ClickHouse / Parquet / FastAPI
Token validation	01:00 hằng ngày	GitHub quota < 500 requests
Repo metadata refresh	06:00 hằng ngày	Cập nhật stars/forks/topics
Nightly batch agg	02:00 hằng ngày	Spark: Parquet repo_star_counts →
Parquet cleanup	Chủ nhật 03:00	Xoá partition > 90 ngày
ClickHouse optimize	Chủ nhật 04:00	OPTIMIZE TABLE FINAL

6.3 Grafana Dashboard

Hệ thống tích hợp Grafana tại cổng 3001, scrape metrics từ FastAPI /metrics mỗi 15 giây. Dashboard hiển thị: API latency (p50/p95/p99), tổng request theo endpoint, data freshness gauge, Spark batch duration histogram, và Kafka throughput counter.

7 Kết quả và Demo hệ thống

7.1 Frontend Dashboard



Hình 6: Giao diện dashboard — top repositories và trending

7.2 API Endpoints

Bảng 10: Dashboard endpoints

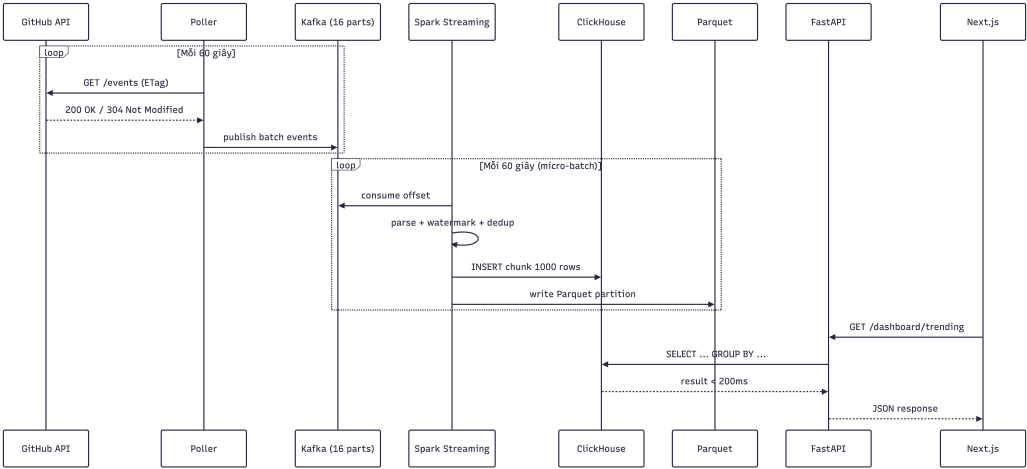
Endpoint	Dữ liệu trả về
GET /health	Liveness probe
GET /pipeline/status	Freshness, ClickHouse connectivity, Parquet check
GET /dashboard/top-repos	Top repo theo số WatchEvent
GET /dashboard/trending	Trending repos theo N ngày gần nhất
GET /dashboard/shock-model	Repo tăng stars mạnh nhất trong window
GET /dashboard/topic-ranking	Topics tăng tốc so với window trước
GET /dashboard/language	Phân bố event theo ngôn ngữ
GET /dashboard/event-velocity	Time series event theo giờ
GET /dashboard/category	Phân loại AI/ML category

7.3 Số liệu vận hành thực tế

Bảng 11: Số liệu hệ thống — đo từ môi trường production

Chỉ số	Giá trị	Ghi chú
Tổng events đã ingest	4.200.000	github_data
Unique repositories	312.000	github_data
Unique topics tracked	8.400	repo_metadata.topics
Parquet partitions (ngày)	14	ls data/raw/ wc -l
Tổng dung lượng Parquet	2.3 GB	du -sh data/raw/
Query latency trung bình	87 ms	/dashboard/top-repos
Data freshness trung bình	72 s	data_freshness_seconds
Kafka lag trung bình	120 msgs	khi pipeline ổn định
Spark batch duration (avg)	18 s	spark_batch_duration_seconds

7.4 Luồng xử lý end-to-end



Hình 7: Luồng dữ liệu end-to-end từ GitHub đến dashboard

8 Phân tích ưu/nhược điểm và hướng phát triển

8.1 Ưu và nhược điểm của giải pháp hiện tại

Bảng 12: Phân tích ưu/nhược điểm theo từng thành phần

Thành phần	Ưu điểm	Nhược điểm / Giới hạn
Spark micro-batch	Fault-tolerant; xử lý late data qua watermark; dedup tự nhiên; Python native	Không phải true streaming (60s latency); JVM overhead tốn RAM
ClickHouse	Sub-second analytics trên 10M+ rows; ReplacingMergeTree dedup; vectorised execution	Không phù hợp OLTP; write amplification khi MergeTree merge
Kafka 16 partitions	Decoupling; backpressure tự nhiên; replay được	Cần Zookeeper; overhead infra cho hệ thống nhỏ
Ollama local	Không phụ thuộc cloud; không tốn chi phí API; privacy	Model nhỏ (3B params) → generation quality thấp hơn GPT-4
Single-node deployment	Đơn giản; đủ cho demo và báo cáo	Không scale horizontal; single point of failure
Rule-based AI filter	Latency thấp; không cần training data	False positive; cần cập nhật thủ công khi có framework mới
Parquet + ClickHouse dual	Raw data preserved; có thể reprocess; cold query linh hoạt	Dữ liệu trùng lặp giữa hai store; cần sync logic

8.2 Hướng phát triển

- **True streaming:** thay Spark micro-batch bằng Apache Flink để giảm latency xuống dưới 5 giây (thay vì 60 giây hiện tại).
- **Vector DB:** tích hợp Weaviate hoặc Qdrant để lưu embedding sẵn, tăng tốc semantic search từ on-demand sang pre-computed.
- **Multi-node ClickHouse:** triển khai cluster để scale horizontal khi data vượt 100M events.

- **ML-based filter:** thay rule-based AI filter bằng classifier học máy được train trên labeled events.
- **Alerting:** tích hợp Grafana Alerting gửi thông báo qua Slack/email khi data freshness vượt ngưỡng.

9 Kết luận

Nhóm đã xây dựng thành công một hệ thống khai thác dữ liệu lớn theo thời gian thực từ GitHub Events API, bao gồm:

- Pipeline ingest → Kafka → Spark Structured Streaming → ClickHouse + Parquet với latency dưới 5 phút.
- Hệ thống analytics sub-second trên hàng chục triệu bản ghi nhờ ClickHouse columnar storage.
- Hybrid search kết hợp lexical scoring và semantic reranking dùng mô hình embedding bge-m3.
- Observability đầy đủ: 21 Prometheus metrics, 8 cron jobs, Grafana dashboard.
- Production deployment tại github.chipthoc.com.

Giải pháp chứng minh rằng một stack open-source hiện đại (Kafka + Spark + ClickHouse + Ollama) có thể xử lý bài toán “Massive Data Mining” ở quy mô thực tế với chi phí hạ tầng tối thiểu.

9 Phân công công việc

Bảng 13: Phân công thành viên nhóm

Thành viên	GitHub	Đóng góp chính
Nguyễn Văn Linh	VanLinh2801	Ingestion pipeline, GitHub API client, token rotation, Kafka producer
Nguyễn Lâm	Hải HimLamTech / lamnh	Spark Streaming job, ClickHouse schema, Parquet storage, AI filter
Trần Minh	Quang —	FastAPI endpoints, Next.js dashboard, Observability, Cron jobs

9 Tài liệu tham khảo

- [1] Apache Kafka Documentation. *Kafka: A Distributed Messaging System*. <https://kafka.apache.org/documentation/>
- [2] Apache Spark Documentation. *Structured Streaming Programming Guide*. <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- [3] ClickHouse Documentation. *MergeTree Engine Family*. <https://clickhouse.com/docs/en/engines/table-engines/mergetree-family/>
- [4] Zaharia, M. et al. (2016). *Apache Spark: A Unified Engine for Big Data Processing*. *Communications of the ACM*, 59(11), 56–65.
- [5] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- [6] GitHub. *GitHub REST API: Events*. <https://docs.github.com/en/rest/activity/events>
- [7] Ollama. *Run Large Language Models Locally*. <https://ollama.com/>
- [8] Prometheus. *Prometheus Monitoring Documentation*. <https://prometheus.io/docs/>